

SAPIENZA UNIVERSITÀ DI ROMA

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica

Corso di Laurea Magistrale in Ingegneria Informatica — Basi di Dati / Data Management

MANUALE TECNICO DI PROGETTO & PREPARAZIONE ORALE

Architettura Neo4j, Modello a Grafo e Query Cypher in Agreeo

Studio approfondito sui fondamenti dei Graph Databases: Index-Free Adjacency, algebra dei pattern Cypher, transazioni ACID concorrenti e ottimizzazione di query complesse su grafi bipartiti su larga scala.

CANDIDATI / AUTORI

Antonio Rubino (Matr. 2226252)

Nicola Moscufo (Matr. 2226190)

DOCENTE / TUTOR DI RIFERIMENTO

Dr. Roberto Maria Delfino

ANNO ACCADEMICO

2025 / 2026

Agreeo Project — Repository Ufficiale: github.com/screamkface/agreeo | Versione Documento: 3.2.0

Indice Generale del Manuale

1. Il Modello del Grafo di Agreeo & Fondamenti dei Graph Database

- 1.1 Dal Modello Relazionale (RDBMS) al Property Graph Model
- 1.2 Il Principio Fisico Cardine: Index-Free Adjacency (IFA)
- 1.3 Architettura Dual-Catalog (TMDB vs MovieLens) e Isolamento dei Concetti
- 1.4 Catalogo Completo dei Nodi, Identificatori e Proprietà di Storage
- 1.5 Catalogo Completo delle Relazioni, Direzionalità e Vincoli di Integrità

2. Sintassi Cypher & Pattern Algebra: Da Zero ad Avanzato

- 2.1 Grammatica Visiva ASCII-Art e Semantica dei Pattern
- 2.2 Manipolazione Dati: MATCH vs CREATE vs MERGE
- 2.3 Upsert Atomico con ON CREATE SET e ON MATCH SET
- 2.4 La Clausola WITH come Barriera di Cardinalità e Pipeline di Trasformazione
- 2.5 OPTIONAL MATCH, Prodotto Cartesiano e Gestione dei NULL
- 2.6 Pattern Negation ad Alte Prestazioni (WHERE NOT)
- 2.7 Manipolazione di Collezioni: UNWIND, collect(), reduce(), coalesce()
- 2.8 La "Trappola Fatale" di UNWIND [] e la Strategia Difensiva di Agreeo
- 2.9 Subquery CALL { WITH ... } e Transazioni Batch in TRANSACTIONS OF n ROWS

3. Architettura Backend & Gestione delle Transazioni ACID

- 3.1 Il Driver neo4j-driver 5.28 e il Protocollo Binario Bolt
- 3.2 Gestione del Connection Pool e Dimensionamento dei Parametri
- 3.3 Le 3 Modalità di Esecuzione: Auto-commit, Managed Read, Managed Write
- 3.4 Meccanismo di Rollback Atomico e Retry su TransientException
- 3.5 Concurrency Control & Pessimistic Locking sui Nodi in Cypher
- 3.6 Parametrizzazione Rigorosa e Protezione Assoluta da Cypher Injection

4. Analisi Linea per Linea delle 6 "Query Regina" di Agreeo

- 4.1 Query I09: Ingestion Batch Massiva di 100k+ Rating (LOAD CSV + Batching)
- 4.2 Query M01: Upsert Idempotente Film TMDB e Sincronizzazione Generi
- 4.3 Query M06: Incremento Atomico Quota Giornaliera di Swipe
- 4.4 Query M23: Collaborative Filtering Pesato a 4 Hop con Formule Matematiche
- 4.5 Query S01 & S06: Grafo Sociale, Amicizie Bidirezionali e Pattern Negation
- 4.6 Query S44: Concorrenza, Locking ed Elezione Vincitore in Movie Night

5. Query Engineering, Profiling ed Ottimizzazione

- 5.1 EXPLAIN vs PROFILE: Analisi dei Piani di Esecuzione Cost-Based
- 5.2 Cosa Misurano Fisicamente i DbHits nello Storage Engine
- 5.3 Gli Operatori Principali: NodeIndexSeek, Expand(All), Filter, Top
- 5.4 Killer Prestazionali: L'Operatore Eager e i Prodotti Cartesiani
- 5.5 Strategia di Indicizzazione: Constraint di Unicità vs Indici B-Tree

6. Le 10 Domande Trabocchetto dell'Orale con Risposte Perfette

Simulazione di colloquio con il Dr. Delfino su Scelte Progettuali, Transazioni e Cypher

Capitolo 1: Il Modello del Grafo di Agreeo & Fondamenti dei Graph Database

1.1 Dal Modello Relazionale (RDBMS) al Property Graph Model

Nei database relazionali (RDBMS come PostgreSQL o MySQL), le relazioni tra entità eterogenee non sono elementi fisici di primo livello (*first-class citizens*), bensì **costrutti logici** calcolati a runtime tramite il confronto tra coppie di valori (Foreign Key → Primary Key). Quando un'applicazione richiede l'attraversamento di pattern a molteplici livelli di profondità (ad esempio: "Trova tutti gli amici dei miei amici che hanno votato positivamente film dello stesso genere che io non ho ancora visto"), il motore relazionale è costretto ad eseguire una cascata di costose operazioni di **JOIN**.

In un RDBMS, ogni **JOIN** richiede l'ispezione di indici globali (tipicamente alberi B-Tree) o la costruzione in memoria di tabelle hash (*Hash Join*) con complessità computazionale proporzionale alla dimensione totale della tabella o degli indici: $O(k \cdot \log N)$, dove N è il numero totale di righe nella tabella target e k è la profondità di attraversamento. Al crescere di N , la cardinalità intermedia esplode esponenzialmente, portando a colli di bottiglia drammatici su memoria e I/O disco.

Il Labeled Property Graph (LPG) di Neo4j rivoluziona questo paradigma introducendo nodi, relazioni e proprietà sia sui vertici sia sugli archi. Le relazioni sono persistite fisicamente sul disco sotto forma di puntatori di memoria bidirezionali diretti.

1.2 Il Principio Fisico Cardine: Index-Free Adjacency (IFA)

Il concetto teorico e implementativo più importante da padroneggiare per l'esame è l'**Index-Free Adjacency (Adiacenza Senza Indici)**.

Definizione Teorica Formale: Index-Free Adjacency (IFA)

Un database a grafo implementa l'Index-Free Adjacency se ciascun nodo memorizza riferimenti diretti (puntatori fisici di memoria o offset di blocco sul disco) ai suoi nodi adiacenti. Di conseguenza, l'attraversamento di una relazione da un nodo A a un nodo B **non richiede alcuna consultazione di indici globali**, ma si riduce alla dereferenziazione diretta di un puntatore.

Confronto di Complessità Computazionale per Hop di Attraversamento

RDBMS (Foreign Key Index Join): $O(\log N)$ vs Neo4j (Index-Free Adjacency): $O(\text{degree}(v))$

Dove N è il numero totale di record presenti nell'intero database, e $\text{degree}(v)$ è il numero di relazioni collegate localmente al singolo nodo v in analisi.

Conseguenza Architettonica: In Neo4j, il tempo necessario per attraversare il grafo da un utente verso i suoi film preferiti è *strettamente indipendente* dal fatto che il database contenga 10.000 nodi oppure 100.000.000 di nodi. La complessità dipende esclusivamente dalla dimensione del vicinato locale del nodo corrente (*local graph density*).

1.3 Architettura Dual-Catalog (TMDB vs MovieLens)

Uno dei punti cardine più apprezzati della progettazione di Agreeo è la separazione netta tra il **catalogo canonico visuale dell'applicazione (TMDB)** e il **catalogo analitico storico di collaborative filtering (MovieLens)**:

- `(:Movie {tmdbId: 550, title: 'Fight Club', ...})` : Rappresenta l'entità canonica dell'app. Contiene locandine, trailer, cast, registi, sinossi e generi TMDB.
- `(:MovieLensMovie {movieLensId: 2959, title: 'Fight Club (1999)', ...})` : Rappresenta il nodo grezzo importato dal dataset MovieLens 100k/32M con tutti i rating storici.

I due mondi sono collegati dalla relazione esplicita `(:MovieLensMovie)-[:MATCHES_TMDB]->(:Movie)`, materializzata tramite l'ingestion di `links.csv`.

```
[ ARCHITETTURA DEL DOPPIO CATALOGO IN AGREEO ]

(:MovieLensUser) ==[RATED {rating, timestamp}]==> (:MovieLensMovie)
      |
      [ :MATCHES_TMDB ]
      |
      v
(:AppUser) =====[LIKED|DISLIKED|WATCHLISTED]=====> (:Movie {tmdbId})
      |
      [ :IN_GENRE ]
      |
      v
      (:Genre)
```

Perché questa scelta architetturale? (Motivazione da Orale)

1. **Disaccoppiamento degli Identificatori:** Impedisce collisioni tra chiavi primarie non coordinate (es. TMDB ID 100 ≠ MovieLens ID 100).
2. **Risoluzione N:1:** Più release MovieLens possono convergere sullo stesso film canonico TMDB senza duplicare le interazioni utente.
3. **Separazione dei Cicli di Vita:** TMDB viene aggiornato dinamicamente via API REST; MovieLens è un dataset batch immutabile. Se TMDB cambia un ID o arricchisce un titolo, la matrice dei rating storici non viene intaccata.

1.4 Catalogo Completo dei Nodi in Agreeo

Label del Nodo	Identificatore Primario	Proprietà Chiave Saliienti	Ruolo Funzionale nel Sistema
AppUser	uid (UUID / Firebase)	emailNormalized, displayName, createdAt, avatarUrl	Utente reale dell'applicazione, autenticato via JWT. Possiede interazioni, liste e amici.
Movie	tmdbId (Integer)	title, releaseDate, runtime, posterPath, movieLensAvgRating	Entità film canonica proiettata nella UI dell'applicazione.
MovieLensMovie	movieLensId (Integer)	title, imdbId, movieLensRatingCount, movieLensAvgRating	Record grezzo del dataset MovieLens, agganciato ai rating storici collaborativi.
MovieLensUser	movieLensUserId (Int)	movieLensUserId (senza email o password)	Utente fittizio storico. Non può mai effettuare login; serve solo come pivot collaborativo.
Genre	name (String UNIQUE)	name (es. 'Action', 'Drama', 'Sci-Fi')	Nodo normalizzato condiviso tra generi TMDB, MovieLens e preferenze esplicite utente.
Tag	name (String UNIQUE)	name, idf, frequency, embedding (384D)	Tag semantico normalizzato associato ai film e arricchito con embedding vettoriali.
DailySwipeQuota	key (uid:YYYY-MM-DD)	uid, day, used, createdAt	Tracciamento atomico e concorrente del limite di swipe giornalieri dell'utente (UTC).
MovieNight	id (UUID)	title, status, round, winnerMovieId, updatedAt	Sessione di voto sincronizzata di gruppo tra amici con gestione di round e tie-breaker.
RecommendationBatch	id (UUID)	kind, createdAt, expiresAt, servedCount	Snapshot persistito delle raccomandazioni servite all'utente per tracciare telemetria e CTR.
InAppNotification	id (UUID)	type, title, body, read, createdAt	Notifiche di sistema, inviti a eventi e richieste di amicizia.

1.5 Catalogo Completo delle Relazioni in Agreeo

Tipo Relazione	Pattern (Sorgente → Destinazione)	Proprietà sull'Arco	Semantica e Concorrenza
LIKED	(:AppUser)-[:LIKED]->(:Movie)	createdAt, updatedAt	Swipe a destra. Segnale esplicito positivo primario (+3.0 nel collaborative).
DISLIKED	(:AppUser)-[:DISLIKED]->(:Movie)	createdAt, updatedAt	Swipe a sinistra. Segnale negativo; attiva la penalizzazione dei generi affini.
SELECTED_FAVORITE	(:AppUser)-[:SELECTED_FAVORITE]->(:Movie)	weight (default 4.0), createdAt	Film del cuore selezionati durante l'onboarding iniziale.
WATCHLISTED	(:AppUser)-[:WATCHLISTED]->(:Movie)	createdAt, updatedAt	Film salvato per visione futura (+1.25 nel collaborative).
ALREADY_SEEN	(:AppUser)-[:ALREADY_SEEN]->(:Movie)	createdAt, updatedAt	Film già visto dall'utente. Escluso da raccomandazioni future.
FRIEND	(:AppUser)-[:FRIEND]-(:AppUser)	createdAt	Amicizia accettata. Interrogata senza direzione in Cypher (grafo non orientato).
SENT_FRIEND_REQUEST	(:AppUser)-[:SENT_FRIEND_REQUEST]->(:AppUser)	requestId, status, createdAt	Richiesta di amicizia pendente ('pending' 'accepted' 'declined').
BLOCKED	(:AppUser)-[:BLOCKED]->(:AppUser)	createdAt	Blocco utente. Usato nei filtri di pattern negation sociale (WHERE NOT).
PARTICIPATES_IN	(:AppUser)-[:PARTICIPATES_IN]->(:MovieNight)	status, isHost, joinedAt	Partecipazione a una serata ('invited' 'joined' 'declined').
VOTED_IN	(:AppUser)-[:VOTED_IN]->(:Movie)	eventId, vote, createdAt	Voto su un film in una sessione. Non esiste un nodo Vote intermedio!
HAS_CANDIDATE	(:MovieNight)-[:HAS_CANDIDATE]->(:Movie)	compatibilityScore, eliminated	Shortlist dei film candidati per la serata Movie Night.
RATED	(:MovieLensUser)-[:RATED]->(:MovieLensMovie)	rating (float), timestamp	Giudizio numerico (0.5 - 5.0) del dataset storico MovieLens.
MATCHES_TMDB	(:MovieLensMovie)-[:MATCHES_TMDB]->(:Movie)	createdAt	Ponte logico tra il film MovieLens grezzo e l'entità film canonica TMDB.

Capitolo 2: Sintassi Cypher & Pattern Algebra: Da Zero ad Avanzato

2.1 Grammatica Visiva ASCII-Art e Semantica dei Pattern

Cypher è un linguaggio dichiarativo progettato per esprimere cammini (*paths*) su grafi tramite una notazione visuale diretta:

- `(u:AppUser)` : Rappresenta un nodo con etichetta (*label*) `AppUser` memorizzato nella variabile `u`.
- `-[r:LIKED]->` : Rappresenta un arco orientato uscente di tipo `LIKED` associato alla variabile di relazione `r`.
- `-[r:FRIEND]-` : Rappresenta un arco attraversabile in entrambe le direzioni (grafo non orientato a livello logico).

2.2 Manipolazione Dati: MATCH vs CREATE vs MERGE

La distinzione tra queste tre primitive è fondamentale per garantire l'integrità del database:

- `MATCH` : Cerca pattern preesistenti nel grafo. Se il pattern non esiste, il flusso delle righe si azzerà (zero risultati) e le clausole successive non vengono eseguite.
- `CREATE` : Inserisce incondizionatamente un nuovo nodo o una nuova relazione. *Attenzione:* Se eseguito ripetutamente senza un vincolo di unicità, duplicherà vertici e archi.
- `MERGE` : Operazione di **upsert atomico**. Se il pattern specificato esiste già nel grafo, agisce come un `MATCH` legando la variabile; se non esiste, agisce come un `CREATE` atomico.

2.3 Upsert Atomico con ON CREATE SET e ON MATCH SET

La combinazione di `MERGE` con le clausole condizionali `ON CREATE` e `ON MATCH` consente di inizializzare o aggiornare le proprietà di nodi e archi senza race condition:

```
MERGE (quota:DailySwipeQuota {key: $quotaKey})
ON CREATE SET quota.uid = $uid,
              quota.day = date($day),
              quota.used = 0,
              quota.createdAt = datetime()
ON MATCH SET quota.lastAccessedAt = datetime()
SET quota.used = coalesce(quota.used, 0) + 1
```

2.4 La Clausola WITH come Barriera di Cardinalità e Pipeline

In Cypher, le query non vengono eseguite come blocchi monolitici ma come **pipeline di streaming di record**. La clausola `WITH` svolge tre funzioni vitali:

1. **Isolamento dello Scope:** Qualsiasi variabile non inclusa esplicitamente in `WITH` viene deallocata e dimenticata dal motore, liberando memoria.
2. **Aggregazione Intermedia:** Consente di calcolare metriche intermedie (es. `count()`, `sum()`) e riutilizzarle come filtri successivi nello stesso statement.
3. **Barriera di Cardinalità (Pivoting):** Se una query attraversa un grafo denso, il numero di percorsi intermedi può esplodere in centinaia di migliaia di combinazioni. Inserendo `ORDER BY ... LIMIT K` subito dopo un `WITH`, si tronca il flusso ai migliori *K* elementi *prima* di espandere il grafo al passo successivo.

2.5 OPTIONAL MATCH, Prodotto Cartesiano e Gestione dei NULL

`OPTIONAL MATCH` è l'equivalente Cypher del `LEFT OUTER JOIN` relazionale: se il pattern corrisponde, lega le variabili; se il pattern non corrisponde, valorizza a `null` le variabili senza interrompere il flusso delle righe.

Trabocchetto da Esame: Il Rischio del Prodotto Cartesiano con Più `OPTIONAL MATCH`

Se eseguiamo in sequenza:

```
OPTIONAL MATCH (u) WHERE ALREADY_SEEN u w Movie
OPTIONAL MATCH (u) WHERE RATED_APP u r Movie
RETURN count(w) count(r)
```

Se l'utente ha visto 10 film e recensito 10 film, il Cypher engine genererà un prodotto cartesiano di $10 \times 10 = 100$ record intermedi! Di conseguenza, `count(w)` restituirà erroneamente 100 anziché 10.

Soluzione adottata in Agreeo (es. Query S01): Utilizzare tassativamente `count(DISTINCT w)` e `count(DISTINCT r)` oppure aggregare con `WITH` tra un match e l'altro!

2.6 Pattern Negation ad Alte Prestazioni (WHERE NOT)

Per escludere elementi correlati in Cypher non è necessario eseguire sottoquery complesse. La clausola `WHERE NOT (pattern)` verifica l'assenza di un arco direttamente nei puntatori del nodo in tempo $O(\text{degree})$:

```
// Trova i candidati film escludendo quelli già swippati o visti dall'utente
MATCH (rec:Movie)
WHERE NOT (me)-[:LIKED|DISLIKED|WATCHLISTED|ALREADY_SEEN]->(rec)
```

2.7 Manipolazione di Collezioni: `UNWIND`, `collect()`, `reduce()`

- `collect(expr)`: Funzione di aggregazione che comprime valori scalari provenienti da più righe in un'unica lista ordinata.
- `UNWIND list AS item`: Funzione inversa di `collect()`: trasforma una lista di N elementi in N record individuali della pipeline.
- `reduce(acc = init, x IN list | expr)`: Esegue una riduzione funzionale su una lista (es. per calcolare somme pesate di penalità senza espandere righe).
- `coalesce(val1, val2, ...)`: Restituisce il primo valore non-null (cruciale per gestire proprietà opzionali o valori predefiniti).

2.8 La "Trappola Fatale" di UNWIND [] e la Difesa in Agreeo

ATTENZIONE ORALE: Cosa succede quando fai UNWIND di una lista vuota []?

In Cypher, eseguire `UNWIND [] AS x` su una lista vuota **uccide istantaneamente la riga**: il numero di record generati è esattamente 0. Se all'interno di una query di aggiornamento film eseguiamo `UNWIND $genres AS g` e l'utente invia un film senza generi (`$genres = []`), l'intera query termina con zero righe prodotte e l'aggiornamento del film fallisce silenziosamente!

Come Agreeo risolve questo problema in produzione: In Agreeo non usiamo mai `UNWIND` crudo su liste che possono essere vuote. Usiamo:

1. Il costrutto `FOREACH (g IN $genres | MERGE ...)` (la clausola `FOREACH` ignora no-op le liste vuote senza azzerare il flusso di esecuzione delle righe).
2. Oppure filtri protetti: `CASE WHEN size($genres) > 0 THEN $genres ELSE [] END` uniti a controlli di cardinalità.

2.9 Subqueries con CALL { WITH ... } e Transazioni Batch

Dalla versione 5.x, Neo4j supporta le subquery correlate complete con `CALL { WITH x ... }`. Esse consentono di eseguire calcoli isolati (ad esempio il rilevamento dei generi sgraditi in `M23`) senza inquinare lo spazio delle variabili della query principale.

Inoltre, per operazioni massive di inserimento dati (es. importazione di milioni di rating MovieLens), Cypher fornisce:

```
LOAD CSV WITH HEADERS FROM 'file:///ratings.csv' AS row
CALL {
  WITH row
  MERGE (u:MovieLensUser {movieLensUserId: toInteger(row.userId)})
  MATCH (m:MovieLensMovie {movieLensId: toInteger(row.movieId)})
  MERGE (u)-[r:RATED]->(m)
  SET r.rating = toFloat(row.rating)
} IN TRANSACTIONS OF 10000 ROWS
```

La clausola `IN TRANSACTIONS OF 10000 ROWS` spezza l'importazione in micro-transazioni indipendenti da 10.000 righe ciascuna, effettuando il *commit* periodico. Senza questa clausola, la transazione rimarrebbe aperta accumulando milioni di mutazioni nella memoria Heap della JVM, causando inevitabilmente il crash del server per `java.lang.OutOfMemoryError: Java heap space`.

Capitolo 3: Architettura Backend & Gestione delle Transazioni ACID

3.1 Il Driver neo4j-driver 5.28 e il Protocollo Binario Bolt

La comunicazione tra il backend Node.js di Agreeo e il database Neo4j avviene tramite il driver ufficiale `neo4j-driver` (v5.28.0) sul protocollo binario proprietario **Bolt** (porta predefinita `7687`).

- Bolt è un protocollo stateful, orientato alla connessione su socket TCP multiplexati.
- Utilizza il formato di serializzazione binaria **PackStream** (simile a MessagePack), garantendo un throughput notevolmente superiore e un overhead di banda ridicolo rispetto alle tradizionali API REST / HTTP JSON.

3.2 Gestione del Connection Pool in Neo4jService.js

In `backend/neo4jService.js`, il driver viene istanziato all'avvio come singleton applicativo configurando esplicitamente i parametri di pooling:

```
this.driver = neo4j.driver(uri, neo4j.auth.basic(user, password), {
  maxConnectionPoolSize: 50,           // Massimo 50 connessioni TCP contemporanee
  connectionAcquisitionTimeout: 30000, // Timeout di 30s per ottenere una connessione libera
  maxTransactionRetryTime: 15000,      // Timeout globale di 15s per il retry automatico
});
```

3.3 Le 3 Modalità di Esecuzione: Auto-commit, Managed Read, Managed Write

Modalità	Metodo Driver	Semantica Transazionale	Uso Tipico in Agreeo
Auto-commit Transaction	<code>session.run(query, params)</code>	Singola query autonoma. Nessun retry automatico; commit immediato se non ci sono errori.	Query di lettura semplici, query diagnostiche della console, script di manutenzione all'avvio.
Managed Read Transaction	<code>session.executeRead(tx => ...)</code>	Transazione di sola lettura. Instrada automaticamente le letture sulle read replicas in un cluster causale. Retry automatico trasparente su fallimenti temporanei.	Console di debug Cypher esposta su Flutter (impedisce scritture fraudolente a livello di driver).
Managed Write Transaction	<code>session.executeWrite(tx => ...)</code>	Transazione ACID completa. Acquisisce lock esclusivi sul server leader. In caso di errore JS o violazione vincoli effettua il rollback atomico totale .	Swipe quota check & update (<code>M06</code>), mutazioni interazioni utente, invio voti Movie Night (<code>S44</code>).

3.4 Meccanismo di Rollback Atomico e Retry su TransientException

In una **Managed Write Transaction** (`executeWrite`), Neo4j garantisce l'isolamento e l'atomicità grazie a un pattern a funzione di ordine superiore (*Unit of Work callback*):

```
await session.executeWrite(async (tx) => {
  // 1. Incremento atomico della quota giornaliera
  const quotaRes = await tx.run(M06_QUERY, { uid, quotaKey, day, increment: 1 });
  const used = quotaRes.records[0].get('usedToday').toNumber();

  // 2. Validazione di business logic applicativa
  if (used > DAILY_SWIPE_LIMIT) {
    // Il lancio dell'eccezione JavaScript abortisce la transazione!
    throw new DailySwipeLimitError('Limite giornaliero di swipe superato');
  }

  // 3. Inserimento della relazione LIKED solo se la quota è lecita
  await tx.run(M09_CREATE_INTERACTION, { uid, tmdbId, type: 'LIKED' });
});
```

Spiegazione del Rollback per l'Orale

Se il limite di swipe viene superato, il controller Node.js lancia un'eccezione JavaScript prima che la callback termini con successo. Il driver Neo4j intercetta l'eccezione, non invia il pacchetto Bolt di `COMMIT`, bensì invia immediatamente il comando `ROLLBACK` al server Neo4j. In questo modo, sia l'incremento del contatore in `DailySwipeQuota` sia la relazione film vengono annullati atomicamente: il database torna esattamente allo stato precedente.

3.5 Concurrency Control & Pessimistic Locking sui Nodi in Cypher

Neo4j utilizza un modello di **strict 2-phase locking (2PL)** a livello di nodo e di relazione. Quando una transazione esegue un'operazione di scrittura su un nodo o modifica una delle sue relazioni, il database acquisisce un **lock esclusivo di scrittura (Exclusive Write Lock)** su quel nodo fino al commit o rollback della transazione.

In Agreeo, sfruttiamo questa caratteristica per implementare un **pessimistic lock esplicito** sulle sessioni di voto concorrenti in Movie Night (Query [S44](#)). Aggiornando forzatamente la proprietà timestamp del nodo evento:

```
SET event.updatedAt = datetime()
```

la transazione acquisisce il lock esclusivo sul nodo `(:MovieNight)`. Qualsiasi altro utente che tenti di inviare un voto contemporaneamente per la stessa serata rimarrà in attesa sul lock, serializzando i voti ed eliminando qualsiasi condizione di race condition sul calcolo dei quorum e dei vincitori.

3.6 Parametrizzazione Rigorosa contro Cypher Injection

Tutte le 162 query di Agreeo sono rigidamente parametrizzate. Nessuna query viene mai costruita concatenando stringhe di input utente.

I valori vengono passati come mappa di parametri al driver (`$uid`, `$tmdbId`, `$vote`). Il query planner di Neo4j analizza la sintassi della query una sola volta (generando l'albero di esecuzione e salvandolo nella **Query Plan Cache**), mentre i parametri vengono associati successivamente in memoria come valori tipizzati, impedendo qualsiasi alterazione semantica della logica di esecuzione.

Capitolo 4: Le 6 "Query Regina" di Agreeo Spiegate Riga per Riga

4.1 Query I09: Ingestion Batch Massiva di 100k+ Rating (MovieLens)

File: `backend/scripts/import_movielens.cypher`. Obiettivo: importare i rating da CSV senza saturare la memoria heap del server.

```
1 LOAD CSV WITH HEADERS FROM 'file:///ratings.csv' AS row
2 CALL {
3   WITH row
4   MERGE (u:MovieLensUser {movieLensUserId: toInteger(row.userId)})
5   MATCH (m:MovieLensMovie {movieLensId: toInteger(row.movieId)})
6   MERGE (u)-[r:RATED]->(m)
7   SET r.rating = toFloat(row.rating),
8       r.timestamp = toInteger(row.timestamp),
9       r.ratedAt = datetime({epochSeconds: toInteger(row.timestamp)})
10 } IN TRANSACTIONS OF 10000 ROWS
```

Analisi Tecnica e Domanda da Orale su I09

- **Riga 1 (`LOAD CSV`)**: Esegue lo streaming riga per riga del file CSV, associando ogni riga a una mappa di stringhe `row`.
- **Riga 3 (`WITH row`)**: Importa la variabile `row` all'interno dello scope isolato della subquery `CALL`.
- **Riga 4 (`MERGE u:MovieLensUser`)**: Crea il nodo utente MovieLens solo se non esiste già. È protetto dall'indice univoco `N05` su `movieLensUserId`, garantendo un'operazione $O(1)$.
- **Riga 5 (`MATCH m:MovieLensMovie`)**: *Perché qui si usa MATCH e non MERGE?* (Domanda classica!). Risposta: I film sono già stati creati nel passaggio precedente dall'ingestion di `movies.csv`. Usare `MATCH` anziché `MERGE` riduce drasticamente l'overhead dei lock di creazione ed evita di creare nodi film orfani e privi di titolo qualora nel CSV rating ci fosse un ID film non valido.
- **Riga 6 (`MERGE (u)-[r:RATED]->(m)`)**: Crea l'arco di valutazione. Se l'utente ha più righe per lo stesso film, la relazione viene aggiornata e non duplicata.
- **Riga 10 (`IN TRANSACTIONS OF 10000 ROWS`)**: Esegue il commit periodico ogni diecimila record, liberando i memory buffers e consentendo l'ingestion ininterrotta di file da gigabyte.

4.2 Query M01: Upsert Idempotente Film TMDB & Gestione Generi

File: `backend/movieRepository.js` (funzione `mergeTmdbMovie`). Chiamata durante l'onboarding e ogni volta che un film viene recuperato da TMDB.

```
1 MERGE (m:Movie {tmdbId: $tmdbId})
2 SET m.title = coalesce($title, m.title),
3     m.originalTitle = coalesce($originalTitle, m.originalTitle),
4     m.overview = CASE WHEN $overview <> '' THEN $overview ELSE coalesce(m.overview, '') END,
5     m.runtime = coalesce($runtime, m.runtime),
6     m.voteAverage = coalesce($voteAverage, m.voteAverage),
7     m.genres = CASE WHEN size($genres) > 0 THEN $genres ELSE coalesce(m.genres, []) END,
8     m.tmdbHydrated = coalesce(m.tmdbHydrated, false) OR $tmdbHydrated
9 FOREACH (genreName IN $genres |
10  MERGE (g:Genre {name: genreName})
11  MERGE (m)-[:IN_GENRE]->(g)
12 )
13 WITH m
14 OPTIONAL MATCH (m)-[:stale:IN_GENRE]->(staleGenre:Genre)
15 FOREACH (relationship IN CASE
16  WHEN size($genres) > 0 AND NOT staleGenre.name IN $genres THEN [stale]
17  ELSE []
18 END | DELETE relationship)
```

Eleganza e Idempotenza di M01: Perché questa query è da 30 e Lode?

1. **Non distruttiva:** L'uso di `coalesce($val, m.val)` fa sì che se TMDB non fornisce un campo (o se passiamo un payload parziale), i dati preesistenti nel database non vengano sovrascritti con `null`.
2. **Sincronizzazione Dinamica dei Generi (Righe 9-12):** Crea ed aggancia automaticamente i nodi `Genre` deduplicandoli.
3. **Pruning Sicuro delle Relazioni Obsolete (Righe 14-18):** Come eliminare un genere che TMDB ha rimosso dal film *senza fare una distruttiva DETACH DELETE su tutto*? La query cerca le relazioni esistenti (`OPTIONAL MATCH`) e, solo se la lista `$genres` in input è valorizzata, usa l'idioma Cypher `FOREACH (r IN CASE WHEN ... THEN [stale] ELSE [] END | DELETE r)`. Se il genere non fa più parte della lista, viene rimosso solo quell'arco specifico!

4.3 Query M06: Incremento Atomico Quota Giornaliera di Swipe

File: `backend/movieRepository.js` (funzione `setMovieInteraction`). Eseguita all'interno di `session.executeWrite`.

```
1 MATCH (u:AppUser {uid: $uid})
2 MERGE (quota:DailySwipeQuota {key: $quotaKey})
3 ON CREATE SET quota.uid = $uid,
4             quota.day = date($day),
5             quota.used = 0,
6             quota.createdAt = datetime()
7 MERGE (u)-[:HAS_DAILY_QUOTA]->(quota)
8 SET quota.used = coalesce(quota.used, 0) + $increment
9 RETURN quota.used AS usedToday
```

Analisi Architettuale: Come Funziona la Quota UTC

- Il parametro `$quotaKey` è generato in modo deterministico dal backend come stringa: `${uid}:${utcDate}` (es. `"usr123:2026-09-13"`).
- Grazie al constraint univoco `N12` su `DailySwipeQuota.key`, qualsiasi richiesta concorrente (es. doppio swipe simultaneo da due telefoni con lo stesso account) converge esattamente sullo stesso nodo in memoria.
- Se il nodo quota per quel giorno non esiste, viene creato con `used = 0`; se esiste, viene incrementato atomicamente di `$increment` (1 per nuovi swipe, 0 se il film era già stato contato).
- Se `usedToday > 20`, il backend Node.js rifiuta la richiesta lanciando un errore, scatenando il rollback automatico della transazione e annullando l'incremento.

4.4 Query M23: Collaborative Filtering Pesato a 4 Hop

File: `backend/movieRepository.js` (funzione `getPersonalizedRecommendationCandidates`). È la query algoritmica più importante dell'intero sistema. Attraversa 4 hop nel grafo bipartito:

$$(me:AppUser) \rightarrow (seed:Movie) \leftarrow (seedMl) \leftarrow (similar:MovieLensUser) \rightarrow (recMl) \rightarrow (rec:Movie)$$

```
1 MATCH (me:AppUser {uid: $uid})
2 CALL {
3   WITH me
4   OPTIONAL MATCH (me)-[r:LIKED|DISLIKED|SELECTED_FAVORITE|WATCHLISTED]->(m:Movie)
5   OPTIONAL MATCH (m)-[:MATCHES_TMDB]-(:MovieLensMovie)-[:IN_GENRE]->(g1:Genre)
6   OPTIONAL MATCH (m)-[:IN_GENRE]->(g2:Genre)
7   WITH r, reduce(acc = [], name IN collect(DISTINCT g1.name) + collect(DISTINCT g2.name) |
8     CASE WHEN name IS NULL OR name IN acc THEN acc ELSE acc + name END) AS genreNames
9   UNWIND genreNames AS genreName
10  WITH genreName,
11    sum(CASE WHEN type(r) = 'DISLIKED' THEN 1 ELSE 0 END) AS dislikedCount,
12    count(r) AS totalInteractions
13  RETURN [g IN collect({name: genreName, count: dislikedCount, total: totalInteractions})
14    WHERE g.count >= $dislikedGenreThreshold] AS dislikedGenres
15 }
16 MATCH (me)-[signal:LIKED|SELECTED_FAVORITE|WATCHLISTED]->(seed:Movie)
17 MATCH (seed)-[:MATCHES_TMDB]-(:seedMl:MovieLensMovie)-[:R1:RATED]-(:similar:MovieLensUser)
18 WHERE r1.rating >= 4.0
19 WITH me, dislikedGenres, similar,
20   count(DISTINCT seed) AS overlapCount,
21   sum(
22     CASE type(signal)
23       WHEN 'SELECTED_FAVORITE' THEN coalesce(signal.weight, 4.0)
24       WHEN 'LIKED' THEN 3.0
25       WHEN 'WATCHLISTED' THEN 1.25 ELSE 1.0 END
26     * exp(-0.005 * duration.inDays(coalesce(signal.createdAt, datetime()), datetime()).days)
27     * (toFloat(r1.rating) - 3.0)
28     * (1.0 / sqrt(log(toFloat(coalesce(seed.movieLensRatingCount, 0)) + 10.0)))
29   ) AS similarityScore
30 WHERE similarityScore > 0
31 ORDER BY similarityScore DESC LIMIT toInteger($neighborLimit)
32 MATCH (similar)-[r2:RATED]->(recMl:MovieLensMovie)-[:MATCHES_TMDB]->(rec:Movie)
33 WHERE NOT (me)-[:LIKED|DISLIKED|WATCHLISTED|ALREADY_SEEN|SELECTED_FAVORITE]->(rec)
34 WITH rec, count(DISTINCT similar) AS similarUsers,
35   sum(similarityScore * (r2.rating - 3.0)) / coalesce(sum(abs(similarityScore)), 1.0) AS
weightedPreference
36 WITH rec, similarUsers, weightedPreference,
37   toFloat(similarUsers) / (toFloat(similarUsers) + $supportShrinkage) AS supportWeight
38 WITH rec, similarUsers,
39   (weightedPreference * supportWeight + (coalesce(rec.movieLensAvgRating, 3.2) - 3.0) * (1.0 -
supportWeight))
40   * log(similarUsers + 1.0) * 10.0 AS collaborativeScore
41 RETURN rec.tmbdId AS tmbdId, rec.title AS title, collaborativeScore AS finalScore
42 ORDER BY finalScore DESC LIMIT 80
```

Spiegazione Matematica Approfondita delle Formule di M23

1. Decadimento Esponenziale Temporale (Riga 26):

$$w_{time} = \exp(-0.005 \cdot \Delta days)$$

Uno swipe effettuato oggi pesa 1.0; uno swipe di 100 giorni fa pesa circa 0.606. I gusti recenti dominano su quelli passati.

2. Rating Centrato sul Neutro (Riga 27):

$$r_{centered} = r - 3.0$$

Nel dataset MovieLens i voti vanno da 0.5 a 5.0. Centrando su 3.0 (il voto neutro), un voto di 5.0 vale +2.0 (forte affinità positiva), mentre un voto di 1.0 vale -2.0 (forte affinità negativa).

3. Damping della Popolarità (Riga 28):

$$w_{pop} = 1 / \sqrt{\log(pop + 10)}$$

Se un film è mainstream (es. *Titanic* con 50.000 voti), condividere questo film dà poca informazione sulla reale somiglianza tra due persone. Questo fattore attenua i film iper-popolari favorendo le "gemme nascoste".

4. Barriera di Cardinalità e Pivoting (Riga 31): `ORDER BY similarityScore DESC LIMIT $neighborLimit` (default 50).

Questo è il segreto delle prestazioni di Agreeo! Limitando a 50 vicini top, il secondo match (linea 32) espande al massimo 50×100 rami anziché milioni di combinazioni!

5. Shrinkage Bayesiano (Righe 36-37):

$$supportWeight = N_{similar} / (N_{similar} + \lambda)$$

Se un film è raccomandato da 1 solo utente ($N=1$, con $\lambda=5$), $supportWeight = 1/6 \approx 0.16$: il punteggio viene attirato verso la media globale del film per evitare allucinazioni statistiche. Se $N=50$, $supportWeight = 50/55 \approx 0.91$: il modello si fida pienamente della preferenza personalizzata.

4.5 Query S01 & S06: Grafo Sociale, Amicizie Bidirezionali & Blocchi

File: `backend/socialRepository.js`. Accettazione di una richiesta di amicizia pendente (`S06`) ed elenco amici con pattern negation e privacy (`S01`):

```
// S06: Accettazione esplicita della richiesta
MATCH (me:AppUser {uid: $uid})
MATCH (from:AppUser)-[r:SENT_FRIEND_REQUEST {requestId: $requestId, status: 'pending'}]->(me)
SET r.status = 'accepted', r.updatedAt = datetime()
MERGE (me)-[a:FRIEND]-(from)
ON CREATE SET a.createdAt = datetime()
RETURN from.uid AS friendId
```

```
// S01: Recupero elenco amici con esclusione blocchi e conteggi film
MATCH (me:AppUser {uid: $uid})-[:FRIEND]-(friend:AppUser)
WHERE NOT (me)-[:BLOCKED]->(friend)
      AND NOT (friend)-[:BLOCKED]->(me)
WITH DISTINCT friend
OPTIONAL MATCH (friend)-[:ALREADY_SEEN]->(watched:Movie)
OPTIONAL MATCH (friend)-[:RATED_APP]->(reviewed:Movie)
WITH friend,
      count(DISTINCT watched) AS watchedCount,
      count(DISTINCT reviewed) AS reviewsCount
RETURN {
  id: friend.uid,
  name: coalesce(friend.displayName, friend.email),
  watchedCount: watchedCount,
  reviewsCount: reviewsCount
} AS friend
ORDER BY toLower(friend.displayName) ASC
```

Punti di Forza da Discutere all'Esame

- **Modellazione Archi Bidirezionali:** In Neo4j ogni relazione ha fisicamente una direzione all'atto della creazione, ma le query possono interrogarla omettendo la freccia (`-[:FRIEND]-`). Questo evita di dover inserire due archi distinti `(A)->(B)` e `(B)->(A)`, risparmiando il 50% di memoria sullo storage dei puntatori.
- **Pattern Negation Bidirezionale:** La clausola `WHERE NOT (me)-[:BLOCKED]->(friend) AND NOT (friend)-[:BLOCKED]->(me)` garantisce che se uno dei due utenti ha bloccato l'altro, l'amicizia viene istantaneamente oscurata per entrambi.

4.6 Query S44: Concorrenza, Locking & Votazione Movie Night

File: `backend/socialRepository.js` (funzione `submitVote`). Gestisce la votazione sincrona di gruppo in una sessione Movie Night.

```
1 MATCH (user:AppUser {uid: $uid})-[part:PARTICIPATES_IN]->(event:MovieNight {id: $eventId})
2 MATCH (event)-[rel:HAS_CANDIDATE]->(movie:Movie {tmdbId: $tmdbId})
3 WHERE part.status = 'joined'
4 AND (rel.eliminated IS NULL OR NOT rel.eliminated)
5 MERGE (user)-[v:VOTED_IN {eventId: $eventId}]->(movie)
6 ON CREATE SET v.createdAt = datetime()
7 SET v.vote = $vote,
8     v.updatedAt = datetime(),
9     event.status = 'voting',
10    event.updatedAt = datetime()
11 RETURN movie.tmdbId AS tmdbId LIMIT 1
```

Spiegazione dei Lock Concorrenti (Domanda Chiave di Basi di Dati)

Riga 5 (`MERGE ... VOTED_IN {eventId: $eventId}`): Il voto non è modellato come un nodo intermedio, ma come un arco diretto con proprietà di contesto `eventId`. Questo riduce la cardinalità dei nodi nel grafo e velocizza il calcolo della matrice di gradimento.

Riga 10 (`SET event.updatedAt = datetime()`): Questa scrittura sul nodo `event` agisce come **Pessimistic Exclusive Write Lock** sul nodo della serata! Se 4 amici votano contemporaneamente entro lo stesso millisecondo, le loro transazioni `executeWrite` vengono serializzate su quel nodo. La prima transazione acquisisce il lock, inserisce il voto e rilascia il lock; le successive vengono accodate ed eseguite in ordine senza conflitti di concorrenza (*deadlock avoidance*).

Capitolo 5: Query Engineering, Profiling ed Ottimizzazione

5.1 EXPLAIN vs PROFILE: Analisi dei Piani di Esecuzione Cost-Based

Neo4j include un cost-based query optimizer (CBO) che traduce ogni statement Cypher in un albero fisico di operatori (*Execution Plan*). Per ispezionare il piano, si antepone alla query una di due parole chiave:

Comando	Meccanismo di Funzionamento	Quando Utilizzarlo
<code>EXPLAIN</code>	Genera e mostra il piano di esecuzione senza eseguire la query sul database. Si basa unicamente sulle statistiche interne dei metadati. Non genera I/O disco né mutazioni.	Analisi a freddo, validazione della correttezza degli indici utilizzati, ispezione di query mutative pericolose (es. grandi cancellazioni).
<code>PROFILE</code>	Esegue realmente la query sul grafo, raccogliendo i contatori prestazionali effettivi a runtime: righe passate per ogni operatore, tempo di esecuzione e <code>DbHits</code> esatti.	Profiling prestazionale accurato, benchmarking, individuazione di operatori inefficienti (<code>Eager</code> , <code>CartesianProduct</code>).

5.2 Cosa Misurano Fisicamente i DbHits nello Storage Engine

Definizione di DbHits (Concetto Chiave di Sistemi di Basi di Dati)

Un **DbHit** è l'unità di misura atomica del lavoro compiuto dallo storage engine di Neo4j. Rappresenta una singola operazione primitiva di lettura o scrittura effettuata sui file di record del database:

- Lettura dell'intestazione di un record di nodo (*Node Record Header*);
- Lettura o scrittura di una proprietà nello store delle proprietà (*Property Store*);
- Attraversamento di un puntatore di relazione (*Relationship Record Traversal*);
- Scansione o ricerca all'interno di una pagina di un indice B-Tree (*Index Seek*).

Regola Aurea di Ottimizzazione: Una query ben ottimizzata deve minimizzare il rapporto tra `DbHits` totali e numero di righe restituite nel `RETURN`. Se una query restituisce 10 righe compiendo 500.000 DbHits, significa che il planner sta effettuando una scansione esaustiva non necessaria (*Full Graph Scan*).

5.3 Gli Operatori Principali del Query Engine

Operatore	Comportamento Fisico	Efficienza
<code>NodeIndexSeek</code>	Usa un indice B-Tree per saltare direttamente all'offset del nodo cercato tramite la proprietà.	OTTIMO (O(LOG N))
<code>NodeByLabelScan</code>	Scansiona sequenzialmente tutti i nodi aventi una determinata etichetta (Full Table Scan).	LENTO SU GRANDI TABELLE
<code>Expand(All)</code>	Attraversa i puntatori di memoria fisici delle relazioni collegate al nodo corrente (Index-Free Adjacency).	OTTIMO (O(DEGREE))
<code>Filter</code>	Valuta una condizione booleana sulle proprietà del record in memoria.	VELOCE (CPU MEMORY)
<code>Top</code>	Implementa <code>ORDER BY ... LIMIT K</code> mantenendo un min-heap di dimensione <i>K</i> in memoria.	MOLTO EFFICIENTE
<code>Eager</code>	Operatore Bloccante: Sospende lo streaming e carica tutte le righe a valle in memoria RAM prima di proseguire.	PERICOLOSO (RISCHIO OOM)
<code>CartesianProduct</code>	Combina ogni riga del ramo sinistro con ogni riga del ramo destro ($M \times N$).	CRITICO (CRASH MEMORY)

5.4 Killer Prestazionali: L'Operatore Eager e i Prodotti Cartesiani

Due fenomeni possono degradare severamente le prestazioni di una query Cypher:

1. **L'operatore Eager** : Si verifica quando Cypher rileva un potenziale conflitto di visibilità tra lettura e scrittura (ad esempio quando si modifica un pattern che viene simultaneamente letto nella stessa pipeline). L'operatore Eager costringe il database a bufferizzare tutte le righe nella heap memory, distruggendo i benefici dello streaming. In Agreeo, abbiamo disaccoppiato letture e scritture con opportune clausole `WITH` per eliminare gli operatori Eager dalle query di produzione.
2. **Prodotti Cartesiani Impliciti**: Se si specificano due pattern sconnessi in un unico `MATCH` (es. `MATCH (u:AppUser), (m:Movie)`), Neo4j genererà tutte le combinazioni possibili. La query `M23` previene questo fenomeno garantendo che ogni pattern sia saldamente ancorato a variabili già legate in precedenza.

5.5 Strategia di Indicizzazione in Agreeo

Agreeo definisce 15 constraint e indici formali in `Neo4jService.createConstraints()` eseguiti all'avvio:

- **Unicità e Backing Index**: `N01` (`AppUser.uid`), `N02` (`AppUser.emailNormalized`), `N03` (`Movie.tmbdId`), `N04` (`MovieLensMovie.movieLensId`), `N05` (`MovieLensUser.movieLensUserId`), `N06` (`Genre.name`), `N09` (`MovieNight.id`), `N12` (`DailySwipeQuota.key`). Ognuno di essi genera un backing B-Tree index che rende le ricerche iniziali $O(1)$ / $O(\log N)$.
- **Indici Secondari B-Tree**: `N11` (`RecommendationBatch.createdAt` per la retention dei batch), `N13` (`Movie.title` per la ricerca canonica), `N15` (`Movie.movieLensRatingCount` per il filtraggio rapido della shortlist).

Capitolo 6: Le 10 Domande Trabocchetto dell'Orale (con Risposte Perfette)

D1. Perché avete scelto un Graph Database (Neo4j) invece di un tradizionale RDBMS come PostgreSQL?

"Abbiamo scelto Neo4j perché il dominio applicativo di Agreeo è intrinsecamente basato su relazioni dense e profonde a molti livelli (grafo bipartito utenti-film, grafo sociale di amicizie, sessioni di voto Movie Night). In PostgreSQL, una query come il nostro Collaborative Filtering (Query `M23`) richiederebbe un `JOIN` su 6 tabelle ponte differenti. In un RDBMS, ogni `JOIN` ha un costo computazionale di $O(k \cdot \log N)$ legato alla scansione degli indici B-Tree globali. In Neo4j, grazie all'**Index-Free Adjacency (IFA)**, le relazioni sono persistite fisicamente come doppi puntatori di memoria: l'attraversamento ha una complessità di $O(\text{degree}(v))$, totalmente indipendente dal numero totale di utenti o film presenti nel database. Questo garantisce latenze predicibili sotto i 40ms anche con milioni di nodi."

Punto chiave da pronunciare: Citare la formula di complessità dell'Index-Free Adjacency e il concetto di puntatori fisici su disco/memoria.

D2. Cosa succede se due utenti votano esattamente nello stesso istante nella stessa Movie Night? C'è rischio di race condition?

"No, non c'è alcun rischio di race condition grazie a due meccanismi integrati: 1) L'operazione viene eseguita dentro una **Managed Write Transaction** (`session.executeWrite`) che garantisce isolamento ACID completo; 2) Nella query Cypher `S44` aggiorniamo la proprietà timestamp dell'evento: `SET event.updatedAt = datetime()`. Questa istruzione forza Neo4j ad acquisire un **Exclusive Write Lock** sul nodo (`:MovieNight`). La seconda richiesta concorrente viene messa in attesa dal driver. Se si verificasse un conflitto transitorio di lock (`TransientException`), il driver gestisce automaticamente il retry trasparente fino a 15 secondi, serializzando perfettamente le votazioni."

Punto chiave da pronunciare: Managed Write Transaction + Exclusive Write Lock sul nodo evento + retry trasparente su `TransientException`.

D3. Perché nella query I09 avete inserito ``IN TRANSACTIONS OF 10000 ROWS``? Cosa accadrebbe se la omettete?

"Se omettessimo la clausola `IN TRANSACTIONS OF 10000 ROWS`, Neo4j tenterebbe di caricare tutti i 100.000+ rating del file CSV all'interno di un'unica gigantesca transazione monolitica. Lo stato delle mutazioni, i lock di scrittura e il rollback log verrebbero accumulati interamente nella memoria Heap della Java Virtual Machine (JVM). Questo porterebbe a frequenti pause del Garbage Collector (GC pauses) e inevitabilmente al crash del processo con un `java.lang.OutOfMemoryError: Java heap space`. La clausola divide l'importazione in transazioni indipendenti da 10.000 righe, effettuando il `commit` e svuotando i buffer della memoria periodicamente."

Punto chiave da pronunciare: JVM Heap Memory exhaustion e GC Pause elimination.

D4. Come impedite che un utente aggiri il limite giornaliero di swipe eseguendo richieste HTTP parallele?

"L'integrità è garantita a livello transazionale tramite il nodo `DailySwipeQuota` (Query `M06`): 1) La chiave primaria del nodo è deterministica: `{uid}:{utcDate}` (es. `"usr1:2026-09-13"`), protetta da un **Constraint di Unicità** su Neo4j (`N12`); 2) La query esegue un `MERGE` su questa chiave all'interno di `session.executeWrite`, acquisendo un lock esclusivo di scrittura sul nodo quota; 3) Il contatore `quota.used` viene incrementato. Se il valore supera il limite consentito, il backend Node.js lancia un'eccezione JavaScript che abortisce la transazione, innescando il **Rollback atomico** di Neo4j che cancella sia l'incremento sia l'eventuale relazione creata."

Punto chiave da pronunciare: Chiave UTC deterministica + Constraint univoco + Rollback su eccezione JS.

D5. Nella query M23 (Collaborative Filtering), perché limitate i vicini (`LIMIT 50`) a metà query e non alla fine?

"Si tratta di una **Barriera di Cardinalità** (*Cardinality Barrier*). Il primo attraversamento trova tutti gli utenti MovieLens che hanno apprezzato i film piaciuti a me. Se non ponessimo un limite intermedio, potremmo avere 5.000 utenti vicini. Se ciascuno di essi ha votato 200 film, il secondo match (linea 32) dovrebbe espandere $5.000 \times 200 = 1.000.000$ di percorsi nel grafo, saturando la RAM e degradando i tempi di risposta. Inserendo `ORDER BY similarityScore DESC LIMIT 50`, filtriamo istantaneamente solo i 50 migliori vicini più simili, riducendo l'esplorazione successiva a poche centinaia di rami controllati, consentendo alla query di completare in soli 25 millisecondi."

Punto chiave da pronunciare: Controllo della cardinalità intermedia (Intermediate Cardinality) ed eliminazione dell'esplosione combinatoria.

D6. Nella query delle amicizie (S01), la relazione è `[:FRIEND]` senza frecce. Neo4j memorizza archi non orientati fisicamente?

"No! Fisicamente, nello storage engine di Neo4j, **ogni relazione è memorizzata tassativamente con una direzione** (da un nodo sorgente a un nodo destinazione con puntatori `firstInRel` e `firstOutRel`). Tuttavia, in Cypher, omettere la freccia nel pattern `(me)-[:FRIEND]-(friend)` indica al query planner di attraversare i puntatori della relazione in entrambe le direzioni (sia entranti sia uscenti). Questa è una best practice fondamentale: memorizziamo un solo arco anziché due archi speculari `(A)->(B)` e `(B)->(A)`, dimezzando l'occupazione di memoria e di disco per il grafo sociale."

Punto chiave da pronunciare: Neo4j memorizza sempre con direzione; la sintassi senza freccia esplora entrambi i puntatori fisici in/out.

D7. Che differenza c'è tra un'istruzione `MATCH (m:Movie {tmdbId: 550})` con e senza indice?

"La differenza nell'Execution Plan è abissale: - **Con Constraint/Indice (il nostro caso):** Il cost-based optimizer genera l'operatore `NodeIndexSeek`. Il database cerca la chiave nell'albero B-Tree con complessità $O(\log N)$ e poi salta direttamente all'offset del nodo sul disco. Tempo: < 1 ms; - **Senza Indice:** Il planner è costretto a generare un `NodeByLabelScan` seguito da un operatore `Filter`. Il database deve scansionare sequenzialmente tutti i nodi `:Movie` del database uno per uno e confrontare la proprietà. Con 1 milione di nodi questo causa una catastrofe prestazionale da decine di secondi."

Punto chiave da pronunciare: `NodeIndexSeek` vs `NodeByLabelScan` + `Filter`.

D8. Perché nel modello non avete creato un nodo per ogni Voto (`Vote`), ma avete usato una relazione diretta `[:VOTED_IN]` tra utente e film?

"Questa è una decisione consapevole di **Graph Data Modeling**. Creare un nodo intermedio `(:Vote)` avrebbe triplicato il numero totale di nodi nel database e costretto ogni query di aggregazione a compiere 2 hop anziché 1: `(User)->(Vote)->(Movie)`. Memorizzando invece la relazione diretta `(:AppUser)-[:VOTED_IN {eventId, vote}]->(Movie)`, manteniamo l'Index-Free Adjacency diretta tra utenti e film, riduciamo i `DbHits` di attraversamento e conserviamo il contesto della votazione come proprietà sull'arco."

Punto chiave da pronunciare: Evitare nodi intermedi inutili (Hop Reduction) e sfruttare le proprietà sulle relazioni (Rich Edge Properties).

D9. Che cosa misura esattamente la metrica `DbHits` visibile eseguendo un `PROFILE` in Neo4j?

"Un `DbHit` misura il lavoro computazionale effettivo richiesto allo storage engine di basso livello di Neo4j per servire una query. Ogni volta che il motore deve leggere l'intestazione di un record di nodo, dereferenziare un puntatore di relazione nella catena delle adiacenze, recuperare una stringa o numero dallo store delle proprietà, o consultare una pagina dell'indice B-Tree, viene conteggiato esattamente un `DbHit`. Ottimizzare una query Cypher significa ridurre al minimo i `DbHits` totali necessari per estrarre il result set."

Punto chiave da pronunciare: Operazione atomica sullo storage engine (record header, pointer dereference, property block access).

D10. Come gestisce il driver Neo4j il retry automatico in `session.executeWrite`? Qual è il vincolo sul codice JavaScript?

"Quando più transazioni concorrenti tentano di scrivere sugli stessi nodi o relazioni, Neo4j può generare una `TransientException` (es. deadlock temporaneo o timeout di acquisizione del lock). Il driver ufficiale, grazie a `session.executeWrite`, intercetta automaticamente queste eccezioni temporanee e ritenta l'intera funzione di callback applicando un algoritmo di **exponential backoff con jitter** (fino al tempo massimo `maxTransactionRetryTime: 15000` ms). **Vincolo fondamentale sul codice applicativo:** Poiché la funzione di callback passata a `executeWrite` può essere rieseguita più volte dal driver prima del commit definitivo, la callback **deve essere assolutamente idempotente e priva di effetti collaterali esterni non reversibili** (non si devono inviare email, emettere eventi WebSocket Socket.IO o effettuare chiamate a gateway di pagamento all'interno della callback)."

Punto chiave da pronunciare: `TransientException` + Exponential backoff con jitter + Idempotenza obbligatoria della callback JS (no side-effects esterni).